CODE REVIEW EVALUATION FORM

JavaScript & Express.js | Undergraduate Programming Course

1. SUBMISSION INFORMATION

Course:	ICS 385 Spring 2026	**Section:**	5
Instructor:	Dr. Debasis Bhattacharya	**Semester:**	Spring
Student Name:	April Hope Harris-Torres	**Student ID:**	harrisah
Project Title:	Week 5 - 5d Code Review of Secrets	**Date:**	02/15/26
Reviewer:	April Hope - facilitated by Claude and ChatGPT to help understand code elements	**Review Type:**	Peer / Instructor

2. CODE SUBMISSION DETAILS

Repository URL:	https://github.com/ahopeh/ics385spring2026/week5/3.5 Secrets Project		
Branch:	main	**Commit Hash:**	
Files Reviewed:	index.js, public/index.html, public/secret.html, package.json, solution.js	**Lines of Code:**	around 80 lines when all files are combined

3. CODE OVERVIEW & PURPOSE

Briefly describe the purpose of the submitted code, its main functionality, the Express.js routes implemented, and any middleware or external packages used.

Summary:

The provided code returns a basic Express app that opens to a password-protected webpage where users are prompted to input a password. When the correct password is submitted, the user will be sent to the protected “secret” page.

Express.js routes implemented: the app uses a get route at “/“ which serves the login page (public/index.html) to the user. There is also a post route at “/check” that processes the password submitted by the user and decide if the user should be sent the secret page or returned to the login page.
middlewear used: body-parser to parse URL-encoded form data that’s submitted via the login form. This allows the server to access the user input via req.body in the post request.
external packages used: in package.json “express”: “^4.18.2” means that the project depnds on the Express.js framework version 4.18.2 or newer minor versions within the 4 range.

4. EVALUATION CRITERIA

Rate each criterion on the scale provided. Use the descriptors as guidance. A score of 4 = Excellent, 3 = Proficient, 2 = Developing, 1 = Beginning, 0 = Not Attempted.

Criterion	Description	Score (0–4)	Weight
Code Correctness & Functionality	Application runs without errors; all Express routes return expected responses; edge cases handled.	4	20%

Criterion	Description	Score (0–4)	Weight
Code Structure & Organization	Logical file/folder structure (e.g., routes/, controllers/, models/); separation of concerns; modular design.	2	15%
Naming Conventions & Readability	Variables, functions, and routes use clear, descriptive names following camelCase conventions; consistent formatting.	4	10%
Express.js Best Practices	Proper use of Router, middleware chaining, error-handling middleware, appropriate HTTP methods and status codes.	2	15%
Error Handling & Validation	Input validation present; try/catch or .catch() used; meaningful error messages returned to client.	0	10%
Comments & Documentation	Inline comments explain non-obvious logic; README or header comments describe setup, dependencies, and usage.	0	10%
Security Considerations	No hardcoded secrets; use of environment variables; input sanitization; helmet or CORS configured if applicable.	1	10%
Testing & Reliability	At least basic test cases provided (e.g., using Jest or Supertest); tests cover primary routes and edge cases.	0	10%

Total Weighted Score:	1.90 / 4.00	**Percentage:**	48 %

5. DETAILED FINDINGS — CODE-LEVEL OBSERVATIONS

Document specific issues, bugs, or noteworthy patterns found during the review. Reference file names and line numbers where applicable.

#	File / Line	Severity	Category	Description / Observation
1	solution.js L 14-15	High / Med / Low	Security	Authentication state is stored in a global variable (userIsAuthorized). This means once one user inputs the correct pw and sets this to true, all subsequent users will automatically be granted access until the server restarts.
2	solution.js L14	High / Med / Low	Security	The password (Love Programming) is hardcoded into source. If the source code is ever shared/leaked the password would be exposed.
3	solution.js L25-32	High / Med / Low	Security	There's no session handling or per-user authorization. This means that if this code were published live on the internet, authentication wouldn't be tied to specific client/browser.
4	solution.js ALL	High / Med / Low	Security	There's no protection against brute-force attack here. Without any rate limiting/lockout/delay attackers dould repeatedly post guess to /check.
5	solution.js L25-30	High / Med / Low	Express best practices	Failed login doesn't return with a failed page, instead it just reloads back to the login page without indicating failure.
6		High / Med / Low		
7		High / Med / Low		
8		High / Med / Low		

6. EXPRESS.JS & JAVASCRIPT CHECKLIST

This checklist is filled out looking at the secrets.js code

Check each item that applies to the submitted code. Mark Y (Yes), N (No), or N/A.

Category	Checklist Item	Y / N / N/A
Server Setup	Server listens on a configurable port (e.g., process.env.PORT)	N (uses const port = 3000)
Server Setup	Entry point file is clearly identified (e.g., app.js or server.js)	N
Routing	Routes are organized using express.Router()	N (defined directly on app)
Routing	RESTful conventions followed (GET, POST, PUT/PATCH, DELETE)	Yes, for GET/ POST only
Routing	Route parameters and query strings used correctly	N/A (none used)
Middleware	Body-parser or express.json() configured for request parsing	Y
Middleware	Custom middleware is reusable and well-documented	N
Middleware	Error-handling middleware defined with (err, req, res, next) signature	N
Async/Await	Promises and async/await used correctly (no unhandled rejections)	N/A (none used)
Async/Await	Callback patterns avoided in favor of modern async patterns	N/A
Dependencies	package.json lists all dependencies; no unused packages	Y (express and body-parser)
Dependencies	node_modules excluded via .gitignore	N

Category	Checklist Item	Y / N / N/A
Security	Environment variables managed via .env / dotenv	N
Security	No sensitive data committed to version control	N

7. QUALITATIVE FEEDBACK

Strengths — What does this submission do well?

:

This is a simple and easy to follow app, with logic that works correctly as expected. The password check does function properly, and there's use of middleware that insinuates an understand of Express. It's a pretty readable and straightforward code.

Areas for Improvement — What should the student focus on next?

:

Security! The password for the page is hardcoded into the source code and stored in plain text without any hashing or salt. Therefore it would be plainly visible to anybody with access to that code. Also, the use of a global authorization variable means that once one person successfully logs in, everyone would be able to log in until the server was reset. Also there's no session handling or input validation, and the app is very vulnerable to brute force attacks.

Suggested Learning Resources

:

bcrypt for password hashing and basic session handling in express

8. OVERALL ASSESSMENT

Grade	Range	Description
A / Excellent	90–100%	Code is well-structured, fully functional, secure, and demonstrates mastery of Express.js concepts.
B / Proficient	80–89%	Code works correctly with minor issues; good organization and documentation; some improvements possible.
C / Developing	70–79%	Code runs but has notable gaps in structure, error handling, or best practices; needs revision.
D / Beginning	60–69%	Significant issues with functionality, structure, or documentation; substantial rework required.
F / Incomplete	Below 60%	Code does not compile/run or is largely incomplete; fundamental concepts not demonstrated.

Final Grade Assigned:	C	**Numeric Score:**	75 / 100

9. REQUIRED REVISIONS & ACTION ITEMS

List any mandatory changes the student must complete before resubmission.

#	Action Item	Priority	Due Date
1	Remove the hardcoded password and replace it with a hashed password using something like bcrypt with environment variable storage.	High / Med / Low	
2	Replace global userIsAuthorized variable with session-based authentication using something like express-session	High / Med / Low	
3	Add some basic input validation as well as HTTP status codes for failed logins.	High / Med / Low	
4		High / Med / Low	

10. ACADEMIC INTEGRITY ACKNOWLEDGMENT

By signing below, the reviewer confirms that this evaluation was conducted fairly and objectively. The student acknowledges receipt of this feedback and understands the revisions required.

Reviewer Signature:	April Hope	**Date:**	02/15/26
Student Signature:		**Date:**	
Instructor Signature:		**Date:**	